UAV Aerial Image Segmentation with DeepLabV3-ResNet50

This project implements a semantic segmentation pipeline for UAV (Unmanned Aerial Vehicle) datasets using **DeepLabV3** with a **ResNet50** backbone in PyTorch. The repository includes the training pipeline, visualization scripts, and generates a `submission.csv` for evaluation.

Project Structure & Output Files

Directory Structure

```
.
├── train.py                    # Main training and inference script
└── UAV_dataset/
    ├── train/
    │   ├── imgs/               # 4,000 RGB training images
    │   └── masks/              # 4,000 corresponding grayscale masks
    └── test/
        └── imgs/               # 1,000 test images (no masks provided)
```

Output Files Generated

- `best_model.pth` : Best model weights based on validation loss.
- `submission.csv` : RLE encoded predictions for the test set.
- `./log/train_log.csv` : Epoch-wise training and validation loss records.
- `./fig/training_loss_curve.png` : Visualization of the training curve.
- `./fig/error_analysis_*.png` : Qualitative error map visualizations.

Script Overview (`train.py`)

The main script handles the end-to-end workflow compactly:

- **Preprocessing & Augmentation:** Resizes images to 512×512, applies ImageNet normalization, and utilizes geometric/color augmentations.

- **Model Pipeline:** Initializes the DeepLabV3-ResNet50 architecture (modified for 16 classes).
- **Training Logic:** Manages the training loop using Adam optimizer, StepLR scheduler, and Inverse Frequency Weighted CrossEntropyLoss to handle severe class imbalance. Implements Early Stopping.
- **Inference & TTA:** Loads the best weights, applies Test-Time Augmentation (horizontal flip), and converts final predictions to RLE format.

To run the pipeline:

```
python train.py
```

Data Analysis & Preprocessing

Dataset Exploration

- **Training Set:** 4,000 RGB images and corresponding grayscale masks (pixel values 0-15 representing semantic classes).
- **Test Set:** 1,000 RGB images for submission.

Class Distribution:
As shown below, background classes (sky, ground, buildings) heavily dominate the dataset compared to smaller objects, indicating a severe class imbalance issue.

Example RGB and Ground Truth Mask (785.png):

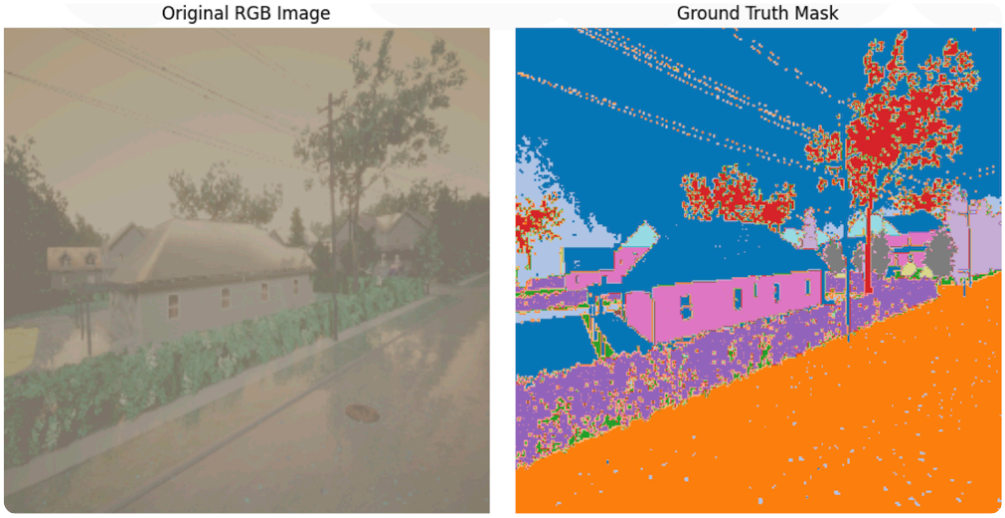


Image Preprocessing & Augmentation

- **Resizing:** All inputs are resized to 512×512. RGB images use Bilinear Interpolation, while Masks use Nearest-Neighbor Interpolation to preserve exact integer class labels (0-15).
- **Normalization:** Z-score standardization using ImageNet statistics (Mean: `[0.485, 0.456, 0.406]` , Std: `[0.229, 0.224, 0.225]`).
- **Online Data Augmentation:**
- **Random Horizontal & Vertical Flip:** Applied with a 50% probability to improve geometric robustness.
- **Color Jitter:** Adjusts brightness, contrast, saturation (0.3), and hue (0.1) to simulate diverse lighting and weather conditions.

Model Selection & Training

Model Architecture

This project utilizes **DeepLabV3** with an ImageNet-pretrained **ResNet50** backbone.

- **ASPP Module:** Atrous Spatial Pyramid Pooling effectively captures multi-scale context, which is crucial for UAV images containing objects of drastically different sizes.
- **Transfer Learning:** The pretrained ResNet50 backbone accelerates convergence and improves generalization on a relatively small dataset (4,000 images).

- **Custom Classification Head:** The final output layer is modified to a 1×1 Conv2d layer (`nn.Conv2d(256, 16, kernel_size=1)`) to output exactly 16 class logits.

Optimization & Loss Function

- **Loss Function:** Due to the extreme class imbalance, a Weighted CrossEntropyLoss is applied using Inverse Frequency Weighting. Weights are calculated as:

$$ \text{weights} \propto \frac{1}{\sqrt{\text{counts}}} $$

- **Optimizer:** Adam (Initial LR = `1e-4`).
- **Scheduler:** StepLR (Step size = 8, Gamma = 0.5) to steadily decrease the learning rate for stable convergence.

Training Setup

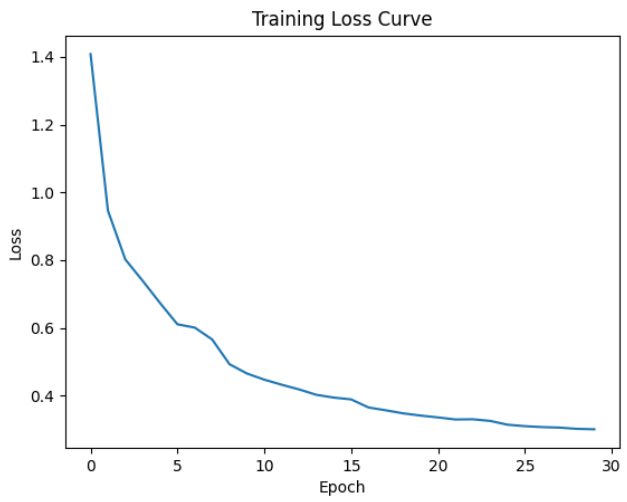
- **Batch Size:** 8 (Optimized for 512×512 input to prevent CUDA OOM errors).
- **Epochs:** 30 Max.
- **Validation Split:** 80/20 (3,200 training / 800 validation).
- **Early Stopping:** Training halts if validation loss does not improve for 5 consecutive epochs.

Experiment & Results

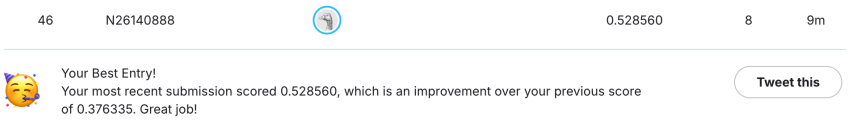
Performance

Training Loss Curve:

The curve demonstrates stable learning and convergence.



Leaderboard mIoU Score: 0.528560



Error Analysis

Qualitative evaluation highlights specific areas where the model struggles:

- **Boundary Blurring:** Borders between adjacent trees or buildings lack sharpness. Power lines in front of trees are completely missed.
- **Lighting/Texture Confusion:** In areas with uneven lighting or high saturation, predictions become localized patches rather than preserving structural integrity.
- **Severe Distortion:** Complex scenes result in ambiguous shapes for buildings and roads, ignoring fine details like windows.

Limitations & Future Improvements

1. **Enhancing Boundary Precision:**

- **Improvement:** Upgrade to **DeepLabV3+** to utilize its additional decoder module for better spatial detail recovery. Alternatively, increase input resolution to 768×768 to preserve fine edges.

2. **Mitigating Small Object Loss:**

- **Improvement:** Implement **Focal Loss** to force the model to focus on hard-to-classify, minority pixels. Expand TTA (Test-Time Augmentation) to include 90-degree and 180-degree rotations.

3. **Resolving Ambiguous Classes:**

- **Improvement:** Increase the Dilation Rate range in the ASPP module to allow the model to rely on a broader global context when distinguishing visually similar classes.